

Custom Buttons

1. Hardware Wiring

This case uses the MSPM0 Robot Expansion Board. The peripherals used do not require additional wiring. You only need to insert the Yahboom MSPM0G3519 core board into the expansion board.

2. Code Explanation

- empty.c

```
int main(void)
{
    USART_Init();

    // Clear timer interrupt flag
    NVIC_ClearPendingIRQ(TIMER_0_INST_INT_IRQN);
    // Enable Timer Interrupt
    NVIC_EnableIRQ(TIMER_0_INST_INT_IRQN);
    // Timer start
    DL_TimerA_startCounter(TIMER_0_INST);

    while (1)
    {
        // Keystroke detection settings in <timer.c>
    }
}
```

The main function executes system initialization and timer enable. In this project, key detection is placed in the <timer.c> file.

- timer.c

```
void TIMER_0_INST_IRQHandler(void)
{
    switch( DL_TimerG_getPendingInterrupt(TIMER_0_INST) )
    {
        case DL_TIMER_IIDX_ZERO:// If it is a 0 overflow interrupt
            Key_Handle();// Key Trigger Detection
            systick_counter++; // +1 per second
            break;

        default:
            break;
    }
}
```

```
uint32_t Get_Time(void)
{
    return systick_counter;
}
```

- TIMER_0_INST_IRQHandler: 1ms timer interrupt function, used for continuous key input detection and counting
- Get_Time: Used to get total count
- key.c

```
// Define key handle
Key_t key1 = {
    .GPIOx = KEY_PORT,
    .GPIO_Pin = KEY_K1_PIN,
    .state = KEY_STATE_RELEASED,
    .presTime = 0,
    .debounceTime = 0
};

/* Debounce time (ms) */
#define DEBOUNCE_DELAY 20 // Typical value: 10-50ms

/**
 * @brief Key scan function (non-blocking)
 * @param key Pointer to KeyHandle
 * @param currentTime Current system time (ms)
 * @param longPressThreshold Long press threshold (ms)
 * @return KeyEvent Returns key event
 */
KeyEvent Key_Scan(Key_t* key, uint32_t currentTime, uint32_t longPressThreshold) {
    // Read pin level: 0=pressed, 1=released
    uint8_t isPressed = 0;

    if(DL_GPIO_readPins(key->GPIOx, key->GPIO_Pin) == 0)
    {
        isPressed = 1;
    }

    switch (key->state) {
        /* State 1: Key released */
        case KEY_STATE_RELEASED:
            if (isPressed) {
                key->state = KEY_STATE_DEBOUNCE; // Enter debounce state
                key->debounceTime = currentTime; // Record debounce start time
            }
            break;

        /* State 2: Debounce checking */
    }
}
```

```

    case KEY_STATE_DEBOUNCE:
        // Check stable state after debounce delay
        if (currentTime - key->debounceTime >= DEBOUNCE_DELAY) {
            if (isPressed) {
                key->state = KEY_STATE_PRESSED;    // Confirm press
                key->pressTime = currentTime;      // Record press time
            } else {
                key->state = KEY_STATE_RELEASED;    // False trigger due to
bounce
            }
        }
        break;

    /* State 3: Pressed */
    case KEY_STATE_PRESSED:
        if (!isPressed) {
            key->state = KEY_STATE_RELEASED;        // Release triggers short
press
            return KEY_EVENT_SHORT;                // Return short press event
        } else if (currentTime - key->pressTime >= longPressThreshold) {
            key->state = KEY_STATE_LONG;            // Trigger long press
            return KEY_EVENT_LONG;                 // Return long press event
        }
        break;

    /* State 4: Long press triggered */
    case KEY_STATE_LONG:
        if (!isPressed) {
            key->state = KEY_STATE_RELEASED;        // Reset state after release
        }
        break;
}

return KEY_EVENT_NONE; // Default: no event
}

void Key_Handle(void)
{
    int16_t LongPressThreshold = 700; // Threshold for long press during key
scanning: 500ms
    static uint32_t lastTick = 0; // initial time
    uint32_t currentTick = Get_Time();
    static int task_flag = 1;

    // Check key every 10ms
    if (currentTick - lastTick >= 10) {
        lastTick = currentTick;

        KeyEvent event = Key_Scan(&key1, currentTick, LongPressThreshold);

        switch (event) {
            case KEY_EVENT_SHORT:

```

```

//          // Handle short press
            printf("short press\r\n");
            LED_Toggle();
            break;
        case KEY_EVENT_LONG:
            // Handle long press

            break;
        default:
            break;
    }
}
}

```

- Key_t key1: Create a key structure to store the configured key pin and press duration.
- Key_Scan: Judge the key press duration by getting the timer count, used for debouncing and determining long press or short press.
- Key_Handle: Key detection processing function. You can change the long press threshold to make long press trigger faster or slower. In the switch event judgment, you can also trigger different functions based on short press or long press. Here, the short press event handles LED level toggle.

- led.c

```

void LED_Toggle(void)
{
    DL_GPIO_togglePins(LED_PORT, LED_D1_PIN);
}

```

- LED_Toggle: LED level toggle. If the light is off, pressing the key will toggle it to on.

3. Program Phenomenon

After burning the program, press the NRST key to reset. Then you can press the K1 key on the expansion board to light up or turn off the D1 labeled LED on the core board.

